

# Les Caractères & Chaîne de Caractères.

La mémoire de l'ordinateur conserve toutes les données sous forme *numérique*. Il n'existe pas de méthode pour stocker directement les caractères. Chaque caractère possède donc son équivalent en code numérique : c'est le *code ASCII* (*American Standard Code for Information Interchange* - traduisez « Code Américain Standard pour l'Echange d'Informations »). Le code ASCII de base représentait les caractères sur *7 bits* (c'est-à-dire 128 caractères possibles, de 0 à 127).

- Les codes 0 à 31 ne sont pas des caractères. On les appelle *caractères de contrôle* (associés à des fonctions de base de MS-DOS), car ils permettent de faire des actions telles que :
  - Retour à la ligne (CR)
  - Bip sonore (BEL)
- Les codes 65 à 90 représentent les majuscules,
- Les codes 97 à 122 représentent les minuscules.

Le *code ASCII* a été mis au point pour la langue anglaise, il ne contient donc pas de caractères accentués, ni de caractères spécifiques à une langue. Pour coder ce type de caractère il faut recourir à un autre code. Il a donc été *étendu* à 8 bits (un octet) pour pouvoir coder plus de caractères (on parle d'ailleurs de *code ASCII étendu*).

Ce code attribue les valeurs 0 à 255 aux lettres majuscules et minuscules, aux chiffres, aux marques de ponctuation et aux autres symboles (caractères accentués dans le cas du code *iso-latin1*). Les deux jeux de caractères ASCII étendus les plus couramment utilisés sont :

- *Le code ASCII étendu OEM*, c'est-à-dire celui qui équipait les premières machines de type IBM PC
- *Le code ASCII étendu ANSI*, utilisé par les systèmes d'exploitation récents

## Remarques

- Il suffit de modifier le 6<sup>ème</sup> bit pour passer de majuscules à minuscules, c'est-à-dire ajouter 32 au code ASCII en base décimale.
- Il existe d'autres codes que le code ASCII pour coder les caractères, et parmi ces codes on trouve :
  - *Le code EBCDIC* (*Extended Binary-Coded Decimal Interchange Code*), développé par IBM, permet de coder des caractères sur 8 bits. Bien que largement répandu sur les machines IBM, il n'a pas eu le succès qu'a connu le code ASCII.
  - *Le code Unicode* est un système de codage des caractères sur 16 bits mis au point en 1991. Le système Unicode permet de représenter n'importe quel caractère par un code sur 16 bits, indépendamment de tout système d'exploitation ou langage de programmation. Il regroupe ainsi la quasi-totalité des alphabets existants (arabe, arménien, cyrillique, grec, hébreu, latin, ...) et est compatible avec le code ASCII.

| Dec | Hx | Oct | Char                        | Dec | Hx | Oct | Html        | Chr | Dec | Hx | Oct | Html    | Chr | Dec | Hx | Oct | Html       | Chr |
|-----|----|-----|-----------------------------|-----|----|-----|-------------|-----|-----|----|-----|---------|-----|-----|----|-----|------------|-----|
| 0   | 0  | 000 | NUL (null)                  | 32  | 20 | 040 | &#32; Space |     | 64  | 40 | 100 | &#64; @ |     | 96  | 60 | 140 | &#96; `    |     |
| 1   | 1  | 001 | SOH (start of heading)      | 33  | 21 | 041 | &#33; !     |     | 65  | 41 | 101 | &#65; A |     | 97  | 61 | 141 | &#97; a    |     |
| 2   | 2  | 002 | STX (start of text)         | 34  | 22 | 042 | &#34; "     |     | 66  | 42 | 102 | &#66; B |     | 98  | 62 | 142 | &#98; b    |     |
| 3   | 3  | 003 | ETX (end of text)           | 35  | 23 | 043 | &#35; #     |     | 67  | 43 | 103 | &#67; C |     | 99  | 63 | 143 | &#99; c    |     |
| 4   | 4  | 004 | EOT (end of transmission)   | 36  | 24 | 044 | &#36; \$    |     | 68  | 44 | 104 | &#68; D |     | 100 | 64 | 144 | &#100; d   |     |
| 5   | 5  | 005 | ENQ (enquiry)               | 37  | 25 | 045 | &#37; %     |     | 69  | 45 | 105 | &#69; E |     | 101 | 65 | 145 | &#101; e   |     |
| 6   | 6  | 006 | ACK (acknowledge)           | 38  | 26 | 046 | &#38; &     |     | 70  | 46 | 106 | &#70; F |     | 102 | 66 | 146 | &#102; f   |     |
| 7   | 7  | 007 | BEL (bell)                  | 39  | 27 | 047 | &#39; '     |     | 71  | 47 | 107 | &#71; G |     | 103 | 67 | 147 | &#103; g   |     |
| 8   | 8  | 010 | BS (backspace)              | 40  | 28 | 050 | &#40; (     |     | 72  | 48 | 110 | &#72; H |     | 104 | 68 | 150 | &#104; h   |     |
| 9   | 9  | 011 | TAB (horizontal tab)        | 41  | 29 | 051 | &#41; )     |     | 73  | 49 | 111 | &#73; I |     | 105 | 69 | 151 | &#105; i   |     |
| 10  | A  | 012 | LF (NL line feed, new line) | 42  | 2A | 052 | &#42; *     |     | 74  | 4A | 112 | &#74; J |     | 106 | 6A | 152 | &#106; j   |     |
| 11  | B  | 013 | VT (vertical tab)           | 43  | 2B | 053 | &#43; +     |     | 75  | 4B | 113 | &#75; K |     | 107 | 6B | 153 | &#107; k   |     |
| 12  | C  | 014 | FF (NP form feed, new page) | 44  | 2C | 054 | &#44; ,     |     | 76  | 4C | 114 | &#76; L |     | 108 | 6C | 154 | &#108; l   |     |
| 13  | D  | 015 | CR (carriage return)        | 45  | 2D | 055 | &#45; -     |     | 77  | 4D | 115 | &#77; M |     | 109 | 6D | 155 | &#109; m   |     |
| 14  | E  | 016 | SO (shift out)              | 46  | 2E | 056 | &#46; .     |     | 78  | 4E | 116 | &#78; N |     | 110 | 6E | 156 | &#110; n   |     |
| 15  | F  | 017 | SI (shift in)               | 47  | 2F | 057 | &#47; /     |     | 79  | 4F | 117 | &#79; O |     | 111 | 6F | 157 | &#111; o   |     |
| 16  | 10 | 020 | DLE (data link escape)      | 48  | 30 | 060 | &#48; 0     |     | 80  | 50 | 120 | &#80; P |     | 112 | 70 | 160 | &#112; p   |     |
| 17  | 11 | 021 | DC1 (device control 1)      | 49  | 31 | 061 | &#49; 1     |     | 81  | 51 | 121 | &#81; Q |     | 113 | 71 | 161 | &#113; q   |     |
| 18  | 12 | 022 | DC2 (device control 2)      | 50  | 32 | 062 | &#50; 2     |     | 82  | 52 | 122 | &#82; R |     | 114 | 72 | 162 | &#114; r   |     |
| 19  | 13 | 023 | DC3 (device control 3)      | 51  | 33 | 063 | &#51; 3     |     | 83  | 53 | 123 | &#83; S |     | 115 | 73 | 163 | &#115; s   |     |
| 20  | 14 | 024 | DC4 (device control 4)      | 52  | 34 | 064 | &#52; 4     |     | 84  | 54 | 124 | &#84; T |     | 116 | 74 | 164 | &#116; t   |     |
| 21  | 15 | 025 | NAK (negative acknowledge)  | 53  | 35 | 065 | &#53; 5     |     | 85  | 55 | 125 | &#85; U |     | 117 | 75 | 165 | &#117; u   |     |
| 22  | 16 | 026 | SYN (synchronous idle)      | 54  | 36 | 066 | &#54; 6     |     | 86  | 56 | 126 | &#86; V |     | 118 | 76 | 166 | &#118; v   |     |
| 23  | 17 | 027 | ETB (end of trans. block)   | 55  | 37 | 067 | &#55; 7     |     | 87  | 57 | 127 | &#87; W |     | 119 | 77 | 167 | &#119; w   |     |
| 24  | 18 | 030 | CAN (cancel)                | 56  | 38 | 070 | &#56; 8     |     | 88  | 58 | 130 | &#88; X |     | 120 | 78 | 170 | &#120; x   |     |
| 25  | 19 | 031 | EM (end of medium)          | 57  | 39 | 071 | &#57; 9     |     | 89  | 59 | 131 | &#89; Y |     | 121 | 79 | 171 | &#121; y   |     |
| 26  | 1A | 032 | SUB (substitute)            | 58  | 3A | 072 | &#58; :     |     | 90  | 5A | 132 | &#90; Z |     | 122 | 7A | 172 | &#122; z   |     |
| 27  | 1B | 033 | ESC (escape)                | 59  | 3B | 073 | &#59; ;     |     | 91  | 5B | 133 | &#91; [ |     | 123 | 7B | 173 | &#123; {   |     |
| 28  | 1C | 034 | FS (file separator)         | 60  | 3C | 074 | &#60; <     |     | 92  | 5C | 134 | &#92; \ |     | 124 | 7C | 174 | &#124;     |     |
| 29  | 1D | 035 | GS (group separator)        | 61  | 3D | 075 | &#61; =     |     | 93  | 5D | 135 | &#93; ] |     | 125 | 7D | 175 | &#125; }   |     |
| 30  | 1E | 036 | RS (record separator)       | 62  | 3E | 076 | &#62; >     |     | 94  | 5E | 136 | &#94; ^ |     | 126 | 7E | 176 | &#126; ~   |     |
| 31  | 1F | 037 | US (unit separator)         | 63  | 3F | 077 | &#63; ?     |     | 95  | 5F | 137 | &#95; _ |     | 127 | 7F | 177 | &#127; DEL |     |

## 19. Les Caractères

Un caractère est une variable de type **Char** qui prend 1 octet (= 8 bits) en mémoire. Lorsqu'on donne une valeur à une variable de type **Char**, celle-ci doit être entre apostrophes ' '. Dans un programme en Pascal, on peut aussi travailler sur un caractère à partir de son numéro dans la table ASCII en utilisant les fonctions **Chr()** ou **#**, ou même une variable String dont on prend un caractère à une position déterminée.

### Exemple

```

Var
Espace, Lettre, Caractere : Char ;
Chaine :String ;
Espace := ' ' ;
Lettre := #80;      { Lettres = 'P' }
Caractere := Chr(102); { Carac = 'f' }
Lettre := Chaine[1] ;

```

Voyons à présent quelques fonctions :

- ✓ **Chr (n)** : Cette fonction renvoie le caractère d'indice n dans la table ASCII. **k:= Chr (64) ;**
- ✓ **Ord (k)** : Cette fonction renvoie l'indice correspondant au caractère k dans la table ASCII. C'est la fonction réciproque de Chr. **i := Ord('M')**
- ✓ **StrUpper(s)** : Convertit une chaîne de caractères minuscules en MAJUSCULES. S'applique aux tableaux de caractères.
- ✓ **StrLower(s)** : Convertit une chaîne de caractères MAJUSCULES en minuscules. S'applique aux tableaux de caractères.
- ✓ **UpCase(k)** : Cette fonction ne s'applique qu'aux caractères seuls (de type Char) pris dans une chaîne s. Convertit un caractère minuscule en MAJUSCULE.
- ✓ **LoCase(k)** : Cette fonction ne s'applique qu'aux caractères seuls (de type Char) pris dans une chaîne s. Convertit un caractère majuscule en MINUSCULE.

### 20. Les Chaînes De Caractères

Le type *String* définit des variables "chaînes de caractères" ayant au maximum 255 signes, ces derniers appartenant à la table ASCII. Une chaîne peut en contenir moins si cela est spécifié lors de la déclaration où le nombre de signes (compris entre 1 et 255) sera mis en crochet à la suite du type String. Le premier caractère de la chaîne a pour indice 1, le dernier a pour indice 255 (ou moins si spécifié lors de la déclaration).

#### Syntaxe

##### Var

```
Chaîne : String ;  
Chaîne : String[50] ;
```

Lorsqu'une valeur est affectée à une chaîne de caractère, on procède comme pour un nombre, mais cette valeur doit être placée entre apostrophes ' ', et si cette valeur contient une apostrophe, alors celle-ci doit être doublée dans notre code.

#### Exemple

##### Var

```
animal : string ;  
  
Animal := 'Chat' ;  
Animal :=1''abeille'  
.
```

#### Remarque

Le type *String* est en fait un tableau à une dimension, dont l'élément d'indice zéro contient une variable de type *char* et dont le rang dans la table ASCII correspond à la longueur de la chaîne. D'une autre manière, la toute première composante [0] comporte le nombre de composantes réellement occupées par les caractères composant la chaîne logée dans ce tableau.

Il est donc possible, de modifier un seul caractère de la chaîne grâce à la **syntaxe** suivante :  
*chaîne[index] :=lettre;*

#### Exemple 1

```
- Si un programme comporte la mention :  
  Var chaîne : String[40] ; on a quelque sorte définit :  
  Var chaîne : Array [1..40] of char ;  
- Si on écrit ensuite :  
  Chaîne := ' Bonjour' ; Alors :  
  Chaîne[0] =7 ;  
  Chaîne[1] ='B' ;
```

### Exemple 2

```
Program Chaine;  
Var nom:String;  
BEGIN  
  
    nom := 'Etiévant' ;  
    nom[2] := 'Z' ;  
    nom[0] := Chr(4) ;  
    WriteLn(nom) ;  
    nom[0] := Chr(28) ;  
    Write(nom, '-tagada') ;  
  
END.
```

L'exemple *Chaine* remplace la deuxième lettre de la variable *nom* en un "Z" majuscule, puis spécifie que la variable ne contient plus que 4 caractères. Ainsi la valeur de la variable *nom* est devenue : *EZié*. Mais après, on dit que la variable *nom* a une longueur de 28 caractères et on s'aperçoit à l'écran que les caractères de rang supérieur à 4 ont été conservés et des espaces ont été insérés pour aller jusqu'à 28 ! Ce qui veut dire que la chaîne affichée n'est pas toujours la valeur totale de la chaîne réelle en mémoire.

Attention cependant aux chaînes déclarées de longueur spécifiée dont la longueur ne doit pas dépasser celle déclarée en début de programme.

## 21. Les Fonctions Relatives Aux Chaînes De Caractères

### 21.1. Accéder A Un Caractère Précis D'une Chaîne De Caractères

Les chaînes de caractères de type *string* du Pascal peuvent être assimilées à des tableaux de caractères. Comme pour ces derniers, il est possible de les indexer en précisant à la fin la position du caractère auquel on souhaite accéder, le premier caractère portant l'index 1. Ainsi, si on considère la chaîne *s* : *String*, pour connaître le 5<sup>e</sup> caractère, on écrira *s*[5] (de type **Char**). L'exemple suivant affiche une chaîne en séparant chaque caractère par un tiret :

```
Var  
    s : String;  
    i : Byte;  
Begin  
    s := 'ABCDEF';  
  
    for i := 1 to Length(s) do  
        Write(s[i], '-');
```

### 21.2. Déterminer La Longueur D'une Chaîne

La plupart des langages informatique comme le *Pascal*, sont dotés en standard de la fonction *Length (S : String) : Byte*, qui permet de connaître la longueur d'une chaîne passée en argument :

```
Length ('bonjour') := 7 ;  
  
for i := 1 to Length(Chaine) do
```

### 21.3. Copier Un Extrait De Chaîne De Caractères

Pour copier une partie d'une chaîne de caractère, on se sert de la fonction : **Copy** (*S : String ; début, longueur : Byte*) qui est de type chaîne (elle renvoie une chaîne), où *début* indique le caractère à partir duquel on souhaite copier la chaîne et *longueur* le nombre de caractères à copier :

```
Var
  s : String;
Begin
  s := Copy('Bonjour', 3, 2);  { s = 'nj' }
End;
```

#### Notes :

- ✓ Si la valeur de *début* est supérieure à la *longueur* de la chaîne, alors la fonction *copy()* renvoie une chaîne vide .
- ✓ Si la valeur de la *longueur* est supérieure au nombres de caractères restant dans la chaîne à partir de la position, alors seuls les caractères présents dans la chaîne seront renvoyés.

### 21.4. Supprimer Une Partie De Chaîne De Caractères

Pour supprimer tout ou partie d'une chaîne, on se sert de la procédure **Delete** (*var S : String; début, longueur : Byte*), où *longueur* indique le nombre de caractères à supprimer en commençant à la position *début*:

```
Var
  s : String;
Begin
  s := 'Bonjour';
  Delete(s, 2, 3);      { s = 'Bour' }
End;
```

### 21.5. Concaténer Deux Ou Plusieurs Chaînes

- ✓ Une première solution est d'utiliser la fonction **Concat** (*s1, s2, ..., sn : String*) : *String* pour ajouter plusieurs chaînes de caractères entre elles :

```
Var
  s : String;
Begin
  s := Concat('Ceci ', 'est ', 'un ', 'exemple. ');
End;
```

- ✓ Une seconde solution est d'utiliser l'opérateur + pour ajouter plusieurs chaînes. Le résultat est exactement identique à **Concat** :

```
Var
  s : String;
Begin
  s := 'Ceci ' + 'est ' + 'un ' + 'exemple.';
End;
```

### 21.6. Connaître La Position D'une Sous-Chaîne Dans Une Chaîne

Pour connaître la position d'une sous-chaîne (qui peut s'avérer n'être qu'un seul caractère) dans une chaîne, il faut se servir de la fonction **Pos (SousChaîne, Chaîne : String) : Byte**. La valeur retournée est l'indice du début de la sous-chaîne dans la chaîne. Si la sous-chaîne n'est pas trouvée, alors Pos renverra **0**.

```
Var
  i : Byte;
Begin
  i := Pos('def', 'abcdefgh');    { i = 4 }
End;
```

#### Note :

**Pos** ne renvoie que la position de la **première occurrence**. Si la sous-chaîne apparaît plusieurs fois, alors les autres occurrences seront ignorées. Pour remédier à ce problème, il faut créer sa propre fonction de recherche.

### 21.7. Autres Fonctions

- ✓ **Insert ( s1, s2, i ) ;**  
Procédure qui insert la chaîne s1 dans la chaîne s2 à la position i.
- ✓ **Str ( x, s ) ;**  
Procédure qui convertit le nombre x (Integer ou Real) en chaîne de caractère de nom s.

```
Var
  s : string;

Begin
  Str(123,s);      { s = '123' }
  Str(1.23,s);     { s = '1.230000000000000E+0001' }
  Str(1.23:6:2,s); { s = ' 1.23' }
  Str(1.23:0:3,s); { s = '1.230' }
End.
```

- ✓ **Val (S :string; X; error:integer) ;**
  - S est la chaîne à convertir. Pour qu'il n'y ait pas d'erreur, il faut que celle-ci contienne un nombre, qu'il soit entier (décimal ou hexadécimal), ou à virgule ;
  - X est la variable qui contiendra le nombre converti. Ce peut être un entier (*ShortInt, Byte, Integer, Word, Longint*), ou bien un réel (*Real, Single, Double, Extended, Comp*) ;

- *error* est le code d'erreur. Si la chaîne *S* contient bien un nombre et que *n* est du type attendu, alors *e* vaut 0. Sinon, *e* correspond à la position de la première erreur dans la chaîne *s* (premier caractère ayant empêché la conversion d'aboutir).

```
Var
  i : Integer;
  w : Word;
  r : Real;
  e : Integer;

Begin
  Val('123',i,e);      { i = 123 ; e = 0 }
  Val('1.23',r,e);     { r = 1.23 ; e = 0 }
  Val('1.2E3',r,e);    { r = 1200 ; e = 0 }
  Val('-123',w,e);     { w = ? ; e = 1 }
  Val('1.23',i,e);     { i = ? ; e = 2 }
  Val('12a',i,e);      { i = ? ; e = 3 }
End.
```

- ✓ FillChar ( *s*, *n*, *i* ) ;  
Procédure qui introduit *n* fois dans la chaîne *s* la valeur *i* (de type Byte ou Char).